

## Cloud Computing (Lab 10)

Name : Tooba Shafique

Reg no : 2023-BSE-065

### Task 1 — GitHub CLI Codespace Setup & Authentication

Goal: Install GH CLI (if not present), authenticate for Codespace use, and connect into the Codespace shell (all steps inside Codespace unless noted).

Install GitHub CLI: `winget install --id GitHub.cli`

```
C:\Users\malik>winget install --id GitHub.cli
The 'msstore' source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service
rly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes [N] No: y
Found GitHub CLI [GitHub.cli] Version 2.83.2
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/cli/cli/releases/download/v2.83.2/gh_2.83.2_windows_amd64.msi
17.7 MB / 17.7 MB
Successfully verified installer hash
Starting package install...
Successfully installed
```

Authenticate GH CLI for Codespaces: `gh auth login -s codespace`

When prompted, generate a GitHub access token (classic). Set scopes to: `admin:org, codespace, repo`.

```

C:\Users\malik>gh auth login -s codespace
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 28CA-4867
Press Enter to open https://github.com/login/device in your browser...
[+] Authentication complete.
- gh config set -h github.com git_protocol https
[+] Configured git protocol
[+] Logged in as toobashafique065

```

List available codespaces: `gh codespace list`

```

C:\Users\malik\Lab10>gh codespace list

```

| NAME                               | DISPLAY NAME      | REPOSITORY             | BRANCH | STATE     | CREATED AT             |
|------------------------------------|-------------------|------------------------|--------|-----------|------------------------|
| fuzzy-bassoon-4jp7p65jj46p37pwv    | fuzzy bassoon     | toobashafique065/Lab9  | main*  | Shutdown  | about 1 day ago        |
| potential-halibut-4jp7p65jqj963... | potential halibut | toobashafique065/Lab10 | main   | Available | less than a minute ago |

Connect to a codespace via SSH: `gh codespace ssh -c <name_of_codespace>` `gh codespace ssh -c potential-halibut-4jp7p65jqj963jgx4`

```

C:\Users\malik\Lab10>gh codespace ssh -c potential-halibut-4jp7p65jqj963jgx4
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

@toobashafique065 ~ /workspaces/Lab10 (main) $

```

## Task 2 — Install AWS CLI, Terraform CLI, Provider Setup

Goal: Install AWS CLI (if not present) & Terraform CLI inside Codespace; configure initial Terraform provider.

A. Install AWS CLI (Skip if already installed)

In Codespace shell, install AWS CLI:

`curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"`

```
@toobashafique065 /workspaces/Lab10 (main) $ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 60.3M  100 60.3M    0     0  135M      0 --:--:-- --:--:-- --:--:-- 135M
@toobashafique065 /workspaces/Lab10 (main) $
```

`unzip awscliv2.zip sudo ./aws/install aws --version`

```
inflating: aws/dist/wheel-0.45.1.dist-info/LICENSE.txt
inflating: aws/dist/wheel-0.45.1.dist-info/RECORD
inflating: aws/dist/wheel-0.45.1.dist-info/REQUESTED
@toobashafique065 /workspaces/Lab10 (main) $ sudo ./aws/install
You can now run: /usr/local/bin/aws --version
@toobashafique065 /workspaces/Lab10 (main) $ aws --version
aws-cli/2.32.24 Python/3.13.11 Linux/6.8.0-1030-azure exe/x86_64.ubuntu.24
@toobashafique065 /workspaces/Lab10 (main) $
```

```
@toobashafique065 /workspaces/Lab10 (main) $ aws configure
AWS Access Key ID [None]: AWS_ACCESS_KEY_ID
AWS Secret Access Key [None]: AWS_SECRET_ACCESS_KEY
Default region name [None]: me-central-1
Default output format [None]: json
@toobashafique065 /workspaces/Lab10 (main) $
```

Then check config files:

`cat ~/.aws/credentials cat`

`~/.aws/config`



```
Preparing to unpack .../terraform_1.14.3-1_amd64.d
Unpacking terraform (1.14.3-1) ...
Setting up terraform (1.14.3-1) ...
/usr/bin/terraform
Terraform v1.14.3
on linux_amd64
```

### C. Provider Configuration (main.tf)

Create main.tf using Vim: `vim main.tf`

Save screenshot as: task2\_provider\_file\_creation.png — creation/editing of main.tf.

Inside main.tf, write: `provider "aws" {`  
`shared_config_files = ["~/.aws/config"]`  
`shared_credentials_files = ["~/.aws/credentials"]`  
`}`

```
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
~
~
~
~
~
~
```

Save and quit: Press ESC then `:wq`

Initialize Terraform: `terraform init`

```
@toobashafique065 @ /workspaces/Lab10 (main) $ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.27.0...
- Installed hashicorp/aws v6.27.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@toobashafique065 @ /workspaces/Lab10 (main) $
```

Show contents of .terraform.lock.hcl: `cat .terraform.lock.hcl`

```
@toobashafique065 @ /workspaces/Lab10 (main) $ cat .terraform.lock.hcl
# This file is maintained automatically by "terraform init".
# Manual edits may be lost in future updates.

provider "registry.terraform.io/hashicorp/aws" {
  version = "6.27.0"
  hashes = [
    "h1:bixp2PSsP5ZG8czGCxcS0n6lF5QF1UXlNroq9cdab4=",
    "zh:177a24b806c72e8484b5cab93b2b38e3d770ae6f745a998b54d6619fd0e8129",
    "zh:4ac4a85c14fb868a3306b542e6a56c10bd6c6d5a67bc0c9b8f6a9060cf5f3be7",
    "zh:552652185bc85c8ba1da1d65dea47c454728a5c6839c458b6dcd3ce71c19ccfc",
    "zh:60284b8172d09aee91eae0856f09855eaf040ce3a58d6933602ae17c53f8ed04",
    "zh:6be38d156756ca61fb8e7c752cc5d769cd709686700ac4b230f40a6e95b5dbc9",
    "zh:7a409138fae4ef42e3a637e37cb9efedf96459e28a3c764fc4e855e8db9a7485",
    "zh:8070cf5224ed1ed3a3e9a59f7c30ff88bf071c7567165275d477c1738a56c064",
    "zh:894439ef340a9a79f69cd759e27ad11c7826adeca27be1b1ca82b3c9702fa300",
    "zh:89d035eebf08a97c89374ff06040955ddc09f275ecca609d0c9d58d149bef5cf",
    "zh:985b1145d724fc1f38369099e4a5087141885740fd6c0b1dbc492171e73c2e49",
    "zh:9b12af85486a96aedd8d7984b0ff811a4b42e3d88dad1a3fb4c0b580d04fa425",
    "zh:a80b47ae8d1475201c86bd94a5dcb9dd4da5e8b73102a90820b68b66b76d50fd",
    "zh:d3395be1556210f82199b9166a6b2e677cee9c4b67e96e63f6c3a98325ad7ab0",
    "zh:db0b869d09657f6f1e4110b56093c5fcd9dbdd97c020db1e577b239c0adcbce",
    "zh:ffc72e680370ae7c21f9bd3082c6317730df805c6797427839a6b6b7e9a26a01",
  ]
}
```

Show contents of .terraform/ directory: `ls .terraform/`

```
@toobashafique065 @ /workspaces/Lab10 (main) $ ls .terraform/
providers
```

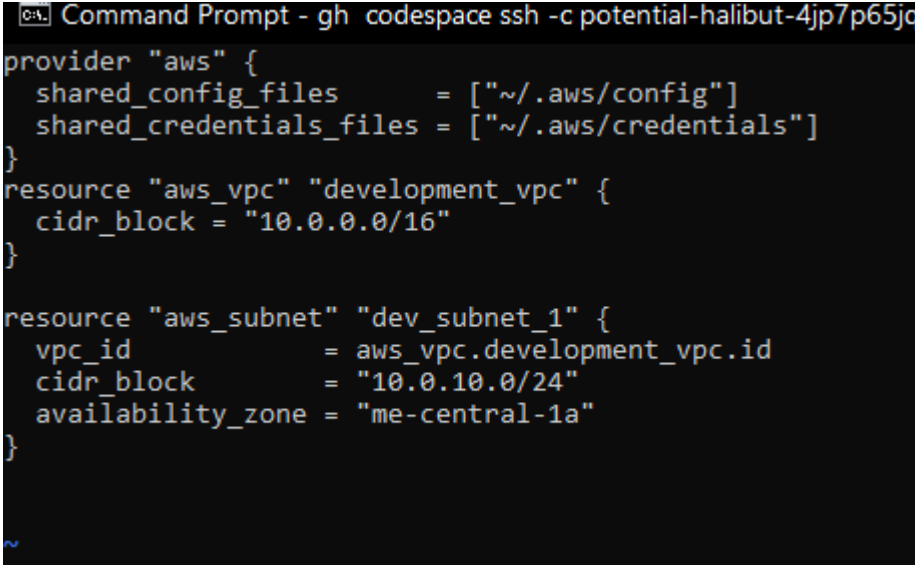
## Task 3 — VPC/Subnet Creation, Initialization, Verification

Goal: Use Terraform to create VPC & Subnet resources, verify via CLI.

Edit main.tf to add:

```
resource "aws_vpc" "development_vpc" { cidr_block  
  
  = "10.0.0.0/16"  
  
}  
  
resource "aws_subnet" "dev_subnet_1" {  
  
  vpc_id    = aws_vpc.development_vpc.id  
  
  cidr_block = "10.0.10.0/24"  availability_zone  
  
  = "me-central-1a"  
  
}
```

Use vim main.tf, edit and insert code.

A screenshot of a terminal window titled "Command Prompt - gh codespace ssh -c potential-halibut-4jp7p65jq". The terminal displays Terraform configuration code for an AWS VPC and a Subnet. The code is as follows:

```
provider "aws" {  
  shared_config_files    = ["~/.aws/config"]  
  shared_credentials_files = ["~/.aws/credentials"]  
}  
  
resource "aws_vpc" "development_vpc" {  
  cidr_block = "10.0.0.0/16"  
}  
  
resource "aws_subnet" "dev_subnet_1" {  
  vpc_id          = aws_vpc.development_vpc.id  
  cidr_block      = "10.0.10.0/24"  
  availability_zone = "me-central-1a"  
}
```

Run: **terraform apply**

Type yes when prompted.

```
Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_vpc.development_vpc: Creating...
aws_vpc.development_vpc: Creation complete after 1s [id=vpc-053de70bf83fd4e18]
aws_subnet.dev_subnet_1: Creating...
aws_subnet.dev_subnet_1: Creation complete after 1s [id=subnet-026072990904c35e4]
```

Save screenshot as: task3\_terraform\_apply\_vpc\_subnet.png — output showing successful apply and resources creation.

Verify resources using AWS CLI:

#### Prerequisites

```
@Sarosh-Majeed → /workspaces/CC_SaroshMajeed_059_Lab10 (main) $ terraform state list
aws_subnet.dev_subnet_1
aws_vpc.development_vpc
```

terraform state show aws\_vpc.development\_vpc look for id = "vpc-053de70bf83fd4e18" terraform state show aws\_subnet.dev\_subnet\_1 look for id = "subnet-026072990904c35e4"

Run: `aws ec2 describe-subnets --filter "Name=subnet-id,Values=<subnet-id>"`



```
@toobashafique065 [ /workspaces/Lab10 (main) ] $ aws ec2 describe-subnets --filter "Name=subnet-id,Values=subnet-019a2d620c4683136"
{
  "Subnets": [
    {
      "AvailabilityZoneId": "mec1-az1",
      "MapCustomerOwnedIpOnLaunch": false,
      "OwnerId": "535942231766",
      "AssignIpv6AddressOnCreation": false,
      "Ipv6CidrBlockAssociationSet": [],
      "SubnetArn": "arn:aws:ec2:me-central-1:535942231766:subnet/subnet-019a2d620c4683136",
      "EnableDns64": false,
      "Ipv6Native": false,
      "PrivateDnsNameOptionsOnLaunch": {
        "HostnameType": "ip-name",
        "EnableResourceNameDnsARecord": false,
        "EnableResourceNameDnsAAAARecord": false
      },
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "SubnetId": "subnet-019a2d620c4683136",
      "State": "available",
      "VpcId": "vpc-0f7ea0a0a5d9d3bb9",
      "CidrBlock": "10.0.10.0/24",
      "AvailableIpAddressCount": 251,
      "AvailabilityZone": "me-central-1a",
      "DefaultForAz": false,
      "MapPublicIpOnLaunch": false
    }
  ]
}
```

Run: `aws ec2 describe-vpcs --filter "Name=vpc-id,Values=<vpc-id>"`

```
@toobashafique065 [ /workspaces/Lab10 (main) ] $ aws ec2 describe-vpcs --filters "Name=vpc-id,Values=vpc-0f7ea0a0a5d9d3bb9"
{
  "Vpcs": [
    {
      "OwnerId": "535942231766",
      "InstanceTenancy": "default",
      "CidrBlockAssociationSet": [
        {
          "AssociationId": "vpc-cidr-assoc-0b00a15ca82cfa9e8",
          "CidrBlock": "10.0.0.0/16",
          "CidrBlockState": {
            "State": "associated"
          }
        }
      ],
      "IsDefault": false,
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "VpcId": "vpc-0f7ea0a0a5d9d3bb9",
      "State": "available",
      "CidrBlock": "10.0.0.0/16",
      "DhcpOptionsId": "dopt-02c0a1e8f3dd284d6"
    }
  ]
}
```

## Task 4 — Data Source, Targeted Destroy, Tags

### A. Data Source & Resource Creation

Add to main.tf: `data "aws_vpc"`

```
"existing_vpc" { default = true
}

resource "aws_subnet" "dev_subnet_1_existing" {

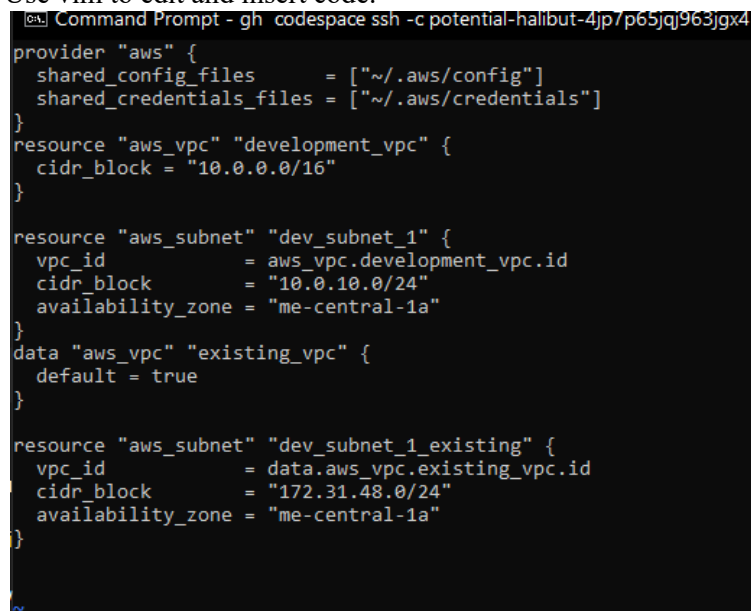
  vpc_id    = data.aws_vpc.existing_vpc.id

  cidr_block = "172.31.48.0/24"  availability_zone =

  "me-central-1a"

}
```

Use vim to edit and insert code.



```
Command Prompt - gh codespace ssh -c potential-halibut-4jp/p65jq963jgx4
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
resource "aws_vpc" "development_vpc" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "dev_subnet_1" {
  vpc_id      = aws_vpc.development_vpc.id
  cidr_block   = "10.0.10.0/24"
  availability_zone = "me-central-1a"
}
data "aws_vpc" "existing_vpc" {
  default = true
}

resource "aws_subnet" "dev_subnet_1_existing" {
  vpc_id      = data.aws_vpc.existing_vpc.id
  cidr_block   = "172.31.48.0/24"
  availability_zone = "me-central-1a"
}
```

Apply configuration: `terraform apply`

Confirm only the new subnet is created.

```
@toobashafique065 [ /workspaces/Lab10 (main) ] $ terraform apply
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development_vpc: Refreshing state... [id=vpc-0f7ea0a0a5d9d3bb9]
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-0b8fa1602193cc7f7]
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-019a2d620c4683136]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# aws_subnet.dev_subnet_1_existing will be created
+ resource "aws_subnet" "dev_subnet_1_existing" {
  + arn                                = (known after apply)
  + assign_ipv6_address_on_creation    = false
  + availability_zone                  = "me-central-1a"
  + availability_zone_id                = (known after apply)
  + cidr_block                         = "172.31.48.0/24"
  + enable_dns64                       = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
  + id                                 = (known after apply)
  + ipv6_cidr_block_association_id     = (known after apply)
  + ipv6_native                        = false
  + map_public_ip_on_launch            = false
  + owner_id                           = (known after apply)
  + private_dns_hostname_type_on_launch = (known after apply)
  + region                             = "me-central-1"
}
```

## B. Targeted Destroy & Refresh

Destroy only one resource: `terraform destroy -target=aws_subnet.dev_subnet_1_existing`

```
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_subnet.dev_subnet_1_existing: Destroying... [id=subnet-033955ec7c0c31790]
aws_subnet.dev_subnet_1_existing: Destruction complete after 0s

Warning: Applied changes may be incomplete

The plan was created with the -target option in effect, so some changes requested in the configuration may have
ignored and the output values may not be fully updated. Run the following command to verify that no other change
pending:
    terraform plan

Note that the -target option is not suitable for routine use, and is provided only for exceptional situations su
recovering from errors or mistakes, or when Terraform specifically suggests to use it as part of an error messag

Destroy complete! Resources: 1 destroyed.
```

Refresh state: `terraform refresh`

```
@toobashafique065 [ /workspaces/Lab10 (main) ] $ terraform refresh
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development_vpc: Refreshing state... [id=vpc-0f7ea0a0a5d9d3bb9]
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-0b8fa1602193cc7f7]
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-019a2d620c4683136]
@toobashafique065 [ /workspaces/Lab10 (main) ] $
```

Re-apply configuration: **terraform apply**

```
+ enable_resource_name_dns_a_record_on_launch = false
+ enable_resource_name_dns_aaaa_record_on_launch = false
+ id = (known after apply)
+ ipv6_cidr_block_association_id = (known after apply)
+ ipv6_native = false
+ map_public_ip_on_launch = false
+ owner_id = (known after apply)
+ private_dns_hostname_type_on_launch = (known after apply)
+ region = "me-central-1"
+ tags_all = (known after apply)
+ vpc_id = "vpc-0b8fa1602193cc7f7"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_subnet.dev_subnet_1_existing: Creating...
aws_subnet.dev_subnet_1_existing: Creation complete after 1s [id=subnet-00fb5b8a40dc8a9ab]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Destroy all resources: **terraform destroy**

```
Plan: 0 to add, 0 to change, 3 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_subnet.dev_subnet_1_existing: Destroying... [id=subnet-00fb5b8a40dc8a9ab]
aws_subnet.dev_subnet_1: Destroying... [id=subnet-019a2d620c4683136]
aws_subnet.dev_subnet_1: Destruction complete after 1s
aws_vpc.development_vpc: Destroying... [id=vpc-0f7ea0a0a5d9d3bb9]
aws_subnet.dev_subnet_1_existing: Destruction complete after 1s
aws_vpc.development_vpc: Destruction complete after 0s

Destroy complete! Resources: 3 destroyed.
```

Plan configuration: **terraform plan**

```
@toobashafique065 @ /workspaces/Lab10 (main) $ terraform plan
data.aws_vpc.existing_vpc: Reading...
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-0b8fa1602193cc7f7]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# aws_subnet.dev_subnet_1 will be created
+ resource "aws_subnet" "dev_subnet_1" {
  + arn                                = (known after apply)
  + assign_ipv6_address_on_creation    = false
  + availability_zone                  = "me-central-1a"
  + availability_zone_id                = (known after apply)
  + cidr_block                         = "10.0.10.0/24"
  + enable_dns64                       = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
  + id                                 = (known after apply)
  + ipv6_cidr_block_association_id     = (known after apply)
  + ipv6_native                        = false
  + map_public_ip_on_launch            = false
  + owner_id                           = (known after apply)
  + private_dns_hostname_type_on_launch = (known after apply)
  + region                             = "me-central-1"
  + tags_all                           = (known after apply)
}
```

Apply again: **terraform apply**

```
+ id                                 = (known after apply)
+ instance_tenancy                   = "default"
+ ipv6_association_id                = (known after apply)
+ ipv6_cidr_block                     = (known after apply)
+ ipv6_cidr_block_network_border_group = (known after apply)
+ main_route_table_id                 = (known after apply)
+ owner_id                           = (known after apply)
+ region                             = "me-central-1"
+ tags_all                           = (known after apply)
}

Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_vpc.development_vpc: Creating...
aws_subnet.dev_subnet_1_existing: Creating...
aws_subnet.dev_subnet_1_existing: Creation complete after 1s [id=subnet-04926bbe6ae615253]
aws_vpc.development_vpc: Creation complete after 2s [id=vpc-046d02bf3a664dac3]
aws_subnet.dev_subnet_1: Creating...
aws_subnet.dev_subnet_1: Creation complete after 1s [id=subnet-0ab1775c37543559a]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

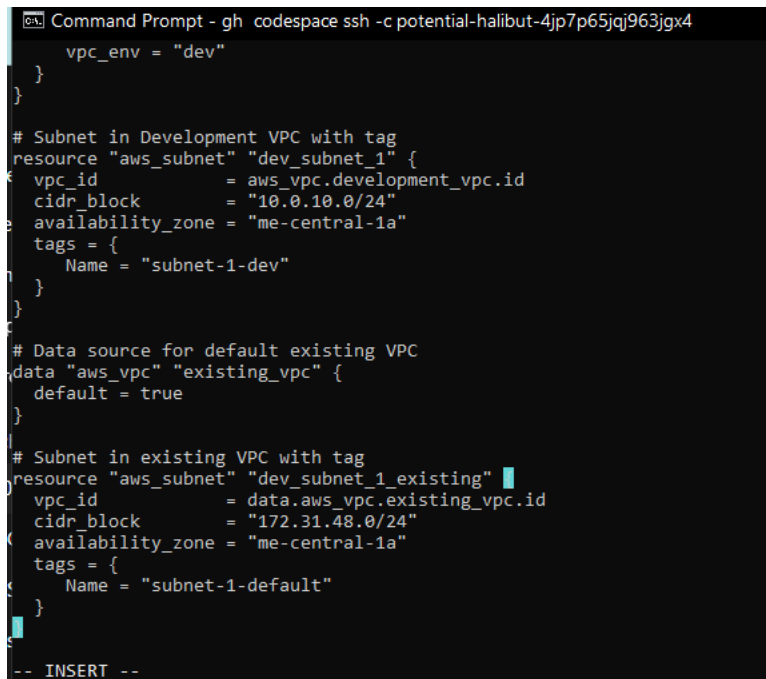
## C. Tagging Resources

Modify main.tf to add tags: **resource**

```
"aws_vpc" "development_vpc" {  
  cidr_block = "10.0.0.0/16"  tags = {  
    Name: "development"  
  }  
  vpc_env = "dev"  
}  
}  
  
resource "aws_subnet" "dev_subnet_1" {  
  vpc_id      = aws_vpc.development_vpc.id  
  cidr_block = "10.0.10.0/24"  availability_zone  
    = "me-central-1a"  
  
  tags = {  
    Name: "subnet-1-dev"  
  }  
}  
  
resource "aws_subnet" "dev_subnet_1_existing" {  
  vpc_id      = data.aws_vpc.existing_vpc.id
```

```
cidr_block = "172.31.48.0/24"  availability_zone =  
  
"me-central-1a"  tags = {  
  
    Name: "subnet-1-default"  
  
}  
}
```

Use vim to update tags.



The screenshot shows a terminal window titled "Command Prompt - gh codespace ssh -c potential-halibut-4jp7p65jqj963jgx4". It displays Terraform configuration code for two subnets. The first subnet, "dev\_subnet\_1", is created in a new VPC with cidr\_block "10.0.10.0/24" and availability\_zone "me-central-1a". The second subnet, "dev\_subnet\_1\_existing", is created in an existing VPC with cidr\_block "172.31.48.0/24" and availability\_zone "me-central-1a". Both subnets have a tag "Name" with the value "subnet-1-dev".

```
Command Prompt - gh codespace ssh -c potential-halibut-4jp7p65jqj963jgx4  
vpc_env = "dev"  
}  
}  
  
# Subnet in Development VPC with tag  
resource "aws_subnet" "dev_subnet_1" {  
  vpc_id      = aws_vpc.development_vpc.id  
  cidr_block   = "10.0.10.0/24"  
  availability_zone = "me-central-1a"  
  tags = {  
    Name = "subnet-1-dev"  
  }  
}  
  
# Data source for default existing VPC  
data "aws_vpc" "existing_vpc" {  
  default = true  
}  
  
# Subnet in existing VPC with tag  
resource "aws_subnet" "dev_subnet_1_existing" {  
  vpc_id      = data.aws_vpc.existing_vpc.id  
  cidr_block   = "172.31.48.0/24"  
  availability_zone = "me-central-1a"  
  tags = {  
    Name = "subnet-1-dev"  
  }  
}  
  
-- INSERT --
```

Run: `terraform refresh terraform`

`apply -auto-approve`

```

groobashafique@ ~ /workspaces/Lab10 (main) $ terraform refresh
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development_vpc: Refreshing state... [id=vpc-046d02bf3a664dac3]
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-0b8fa1602193cc7f7]
aws_subnet.dev_subnet_1_existing: Refreshing state... [id=subnet-04926bbe6ae615253]
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-0ab1775c37543559a]
groobashafique@ ~ /workspaces/Lab10 (main) $ terraform apply -auto-approve
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development_vpc: Refreshing state... [id=vpc-046d02bf3a664dac3]
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-0b8fa1602193cc7f7]
aws_subnet.dev_subnet_1_existing: Refreshing state... [id=subnet-04926bbe6ae615253]
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-0ab1775c37543559a]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  ~ update in-place

Terraform will perform the following actions:

# aws_subnet.dev_subnet_1 will be updated in-place
~ resource "aws_subnet" "dev_subnet_1" {
  id          = "subnet-0ab1775c37543559a"
  ~ tags      = {
    + "Name" = "subnet-1-dev"
  }
  ~ tags_all  = {
    + "Name" = "subnet-1-dev"
  }
  # (20 unchanged attributes hidden)
}

# aws_subnet.dev_subnet_1_existing will be updated in-place
~ resource "aws_subnet" "dev_subnet_1_existing" {
  id          = "subnet-04926bbe6ae615253"
  ~ tags      = {
    + "Name" = "subnet-1-default"
  }
  ~ tags_all  = {

```

Remove `vpc_env = "dev"` tag from `development_vpc` resource, re-plan/apply:

vim main.tf

## Prerequisites

Delete this line : `vpc_env = "dev"` From

this block `resource "aws_vpc"`

```
"development_vpc" { cidr_block =
```

```
"10.0.0.0/16"
```

```
tags = {
```

```
    Name = "development"
```

```
vpc_env = "dev"
```

```
}
```

```
} ESC and :wq
```



### Steps after editing:

1. Save the file.
2. Run: terraform plan
3. Then run: terraform apply -auto-approve

```
ptooobashafique065 @ /workspaces/Lab10 (main) $ terraform plan
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development_vpc: Refreshing state... [id=vpc-046d02bf3a664dac3]
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-0b8fa1602193cc7f7]
aws_subnet.dev_subnet_1_existing: Refreshing state... [id=subnet-04926bbe6ae615253]
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-0ab1775c37543559a]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  ~ update in-place

Terraform will perform the following actions:

# aws_vpc.development_vpc will be updated in-place
~ resource "aws_vpc" "development_vpc" {
  id          = "vpc-046d02bf3a664dac3"
  ~ tags      = {
    "Name"     = "development"
    - "vpc_env" = "dev" -> null
  }
  ~ tags_all  = {
    - "vpc_env" = "dev" -> null
    # (1 unchanged element hidden)
  }
  # (19 unchanged attributes hidden)
}

Plan: 0 to add, 1 to change, 0 to destroy.
```

```

gtoobashafique065 @ /workspaces/Lab10 (main) $ terraform apply -auto-approve
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development_vpc: Refreshing state... [id=vpc-046d02bf3a664dac3]
data.aws_vpc.existing_vpc: Read complete after 0s [id=vpc-0b8fa1602193cc7f7]
aws_subnet.dev_subnet_1_existing: Refreshing state... [id=subnet-04926bbe6ae615253]
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-0ab1775c37543559a]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  ~ update in-place

Terraform will perform the following actions:

  # aws_vpc.development_vpc will be updated in-place
  ~ resource "aws_vpc" "development_vpc" {
    id          = "vpc-046d02bf3a664dac3"
    ~ tags      = {
        "Name" = "development"
        - "vpc_env" = "dev" -> null
      }
    ~ tags_all = {
        - "vpc_env" = "dev" -> null
        # (1 unchanged element hidden)
      }
    # (19 unchanged attributes hidden)
  }

Plan: 0 to add, 1 to change, 0 to destroy.
aws_vpc.development_vpc: Modifying... [id=vpc-046d02bf3a664dac3]
aws_vpc.development_vpc: Modifications complete after 1s [id=vpc-046d02bf3a664dac3]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

```

## Task 5 — State File Inspection & Terraform State Commands

Destroy all resources: `terraform destroy`

```

- default_route_table_id      = "rtb-0ddc98eb306d5222a" -> null
- default_security_group_id   = "sg-095593e9d2f84f17f" -> null
- dhcp_options_id             = "dopt-02c0a1e8f3dd284d6" -> null
- enable_dns_hostnames        = false -> null
- enable_dns_support          = true -> null
- enable_network_address_usage_metrics = false -> null
- id                           = "vpc-046d02bf3a664dac3" -> null
- instance_tenancy            = "default" -> null
- ipv6_netmask_length         = 0 -> null
- main_route_table_id         = "rtb-0ddc98eb306d5222a" -> null
- owner_id                    = "535942231766" -> null
- region                      = "me-central-1" -> null
- tags                        = {
  - "Name" = "development"
} -> null
- tags_all                    = {
  - "Name" = "development"
} -> null
# (4 unchanged attributes hidden)
}

Plan: 0 to add, 0 to change, 3 to destroy.

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_subnet.dev_subnet_1_existing: Destroying... [id=subnet-04926bbe6ae615253]
aws_subnet.dev_subnet_1: Destroying... [id=subnet-0ab1775c37543559a]
aws_subnet.dev_subnet_1: Destruction complete after 1s
aws_vpc.development_vpc: Destroying... [id=vpc-046d02bf3a664dac3]
aws_subnet.dev_subnet_1_existing: Destruction complete after 1s
aws_vpc.development_vpc: Destruction complete after 1s

Destroy complete! Resources: 3 destroyed.

```

Inspect state files after destroy:

Run: `cat terraform.tfstate`

```

@toobashafique065 [main] $ cat terraform.tfstate
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 30,
  "lineage": "085aa91c-daa5-b2aa-fd61-e66f10576ff1",
  "outputs": {},
  "resources": [],
  "check_results": null
}

```

Run: `cat terraform.tfstate.backup`

```

stoobashafique065 @ /workspaces/Lab10 (main) $ cat terraform.tfstate.backup
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 25,
  "lineage": "085aa91c-daa5-b2aa-fd61-e66f10576ff1",
  "outputs": {},
  "resources": [
    {
      "mode": "data",
      "type": "aws_vpc",
      "name": "existing_vpc",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "schema_version": 0,
          "attributes": {
            "arn": "arn:aws:ec2:me-central-1:535942231766:vpc/vpc-0b8fa1602193cc7f7",
            "cidr_block": "172.31.0.0/16",
            "cidr_block_associations": [
              {
                "association_id": "vpc-cidr-assoc-01aed097fc6b6afe4",
                "cidr_block": "172.31.0.0/16",
                "state": "associated"
              }
            ],
            "default": true,
            "dhcp_options_id": "dopt-02c0a1e8f3dd284d6",
            "enable_dns_hostnames": true,
            "enable_dns_support": true,
            "enable_network_address_usage_metrics": false,
            "filter": null,
            "id": "vpc-0b8fa1602193cc7f7",
            "instance_tenancy": "default",
            "ipv6_association_id": "",

```

## Recreate resources: **terraform apply**

```

stoobashafique065 @ /workspaces/Lab10 (main) $ terraform apply -auto-approve
data.aws_vpc.existing_vpc: Reading...
data.aws_vpc.existing_vpc: Read complete after 0s [id=vpc-0b8fa1602193cc7f7]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# aws_subnet.dev_subnet_1 will be created
+ resource "aws_subnet" "dev_subnet_1" {
  + arn                                     = (known after apply)
  + assign_ipv6_address_on_creation         = false
  + availability_zone                       = "me-central-1a"
  + availability_zone_id                   = (known after apply)
  + cidr_block                             = "10.0.10.0/24"
  + enable_dns64                           = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
  + id                                     = (known after apply)
  + ipv6_cidr_block_association_id         = (known after apply)
  + ipv6_native                            = false
  + map_public_ip_on_launch                = false
  + owner_id                              = (known after apply)
  + private_dns_hostname_type_on_launch    = (known after apply)
  + region                                 = "me-central-1"
  + tags                                  = {
    + "Name" = "subnet-1-dev"
  }
  + tags_all                              = {
    + "Name" = "subnet-1-dev"
  }
  + vpc_id                                = (known after apply)
}

```

View state files: Run: **cat terraform.tfstate**

```
@toobashafique065 /workspaces/Lab10 (main) $ cat terraform.tfstate
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 34,
  "lineage": "085aa91c-daa5-b2aa-fd61-e66f10576ff1",
  "outputs": {},
  "resources": [
    {
      "mode": "data",
      "type": "aws_vpc",
      "name": "existing_vpc",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "schema_version": 0,
          "attributes": {
            "arn": "arn:aws:ec2:me-central-1:535942231766:vpc/vpc-0b8fa1602193cc7f7",
            "cidr_block": "172.31.0.0/16",
            "cidr_block_associations": [
              {
                "association_id": "vpc-cidr-assoc-01aed097fc0b0afe4",
                "cidr_block": "172.31.0.0/16",
                "state": "associated"
              }
            ]
          },
          "default": true,
          "dhcp_options_id": "dopt-02c0a1e8f3dd284d6",
          "enable_dns_hostnames": true,
          "enable_dns_support": true,
          "enable_network_address_usage_metrics": false,
          "filter": null,
          "id": "vpc-0b8fa1602193cc7f7",
          "instance_tenancy": "default",
```

Run: `cat terraform.tfstate.backup`

```
@toobashafique065 /workspaces/Lab10 (main) $ cat terraform.tfstate.backup
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 30,
  "lineage": "085aa91c-daa5-b2aa-fd61-e66f10576ff1",
  "outputs": {},
  "resources": [],
  "check_results": null
}
```

List resources: `terraform state lis`

```
@toobashafique065 /workspaces/Lab10 (main) $ terraform state list
data.aws_vpc.existing_vpc
aws_subnet.dev_subnet_1
aws_subnet.dev_subnet_1_existing
aws_vpc.development_vpc
```

Show full attributes: `terraform state show <resource-name>`

```
@toobashafique065 @ /workspaces/Lab10 (main) $ terraform state show aws_vpc.development_vpc
# aws_vpc.development_vpc:
resource "aws_vpc" "development_vpc" {
  arn                                = "arn:aws:ec2:me-central-1:535942231766:vpc/vpc-05764833e3ed15162"
  assign_generated_ipv6_cidr_block   = false
  cidr_block                         = "10.0.0.0/16"
  default_network_acl_id            = "acl-0911efe2c9a5702cf"
  default_route_table_id            = "rtb-0e2dc76238cd6d3eb"
  default_security_group_id         = "sg-0d5bedbdc14b19daf"
  dhcp_options_id                   = "dopt-02c0a1e8f3dd284d6"
  enable_dns_hostnames               = false
  enable_dns_support                 = true
  enable_network_address_usage_metrics = false
  id                                = "vpc-05764833e3ed15162"
  instance_tenancy                   = "default"
  ipv6_association_id               = null
  ipv6_cidr_block                    = null
  ipv6_cidr_block_network_border_group = null
  ipv6_ipam_pool_id                 = null
  ipv6_netmask_length               = 0
  main_route_table_id               = "rtb-0e2dc76238cd6d3eb"
  owner_id                          = "535942231766"
  region                            = "me-central-1"
  tags                              = {
    "Name" = "development"
  }
  tags_all                          = {
    "Name" = "development"
  }
}
```

Note: Do NOT run the following (for information only): `terraform state rm <resource-name>`

Mentioned for knowledge — do not run.

## Task 6 — Terraform Outputs & Attributes Reporting

Add outputs in main.tf: `output "dev-vpc-`

`id" { value =`

`aws_vpc.development_vpc.id`

`}`

`output "dev-subnet-id" { value =`

`aws_subnet.dev_subnet_1.id`

`}`

`output "dev-vpc-arn" { value =`

`aws_vpc.development_vpc.arn }`

```
output "dev-subnet-arn" { value =  
  
aws_subnet.dev_subnet_1.arn  
  
}
```

```
@toobashafique065 [ /workspaces/Lab10 (main) ] $ terraform apply -auto-approve  
data.aws_vpc.existing_vpc: Reading...  
aws_vpc.development_vpc: Refreshing state... [id=vpc-05764833e3ed15162]  
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-0b8fa1602193cc7f7]  
aws_subnet.dev_subnet_1_existing: Refreshing state... [id=subnet-088a3a5a00949bdba]  
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-09b10ebd14209d020]  
  
Changes to Outputs:  
+ dev-subnet-arn = "arn:aws:ec2:me-central-1:535942231766:subnet/subnet-09b10ebd14209d020"  
+ dev-subnet-id  = "subnet-09b10ebd14209d020"  
+ dev-vpc-arn   = "arn:aws:ec2:me-central-1:535942231766:vpc/vpc-05764833e3ed15162"  
+ dev-vpc-id    = "vpc-05764833e3ed15162"  
  
You can apply this plan to save these new output values to the Terraform state, without changing any real  
infrastructure.  
  
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.  
  
Outputs:  
  
dev-subnet-arn = "arn:aws:ec2:me-central-1:535942231766:subnet/subnet-09b10ebd14209d020"  
dev-subnet-id  = "subnet-09b10ebd14209d020"  
dev-vpc-arn   = "arn:aws:ec2:me-central-1:535942231766:vpc/vpc-05764833e3ed15162"  
dev-vpc-id    = "vpc-05764833e3ed15162"  
@toobashafique065 [ /workspaces/Lab10 (main) ] $ terraform output  
dev-subnet-arn = "arn:aws:ec2:me-central-1:535942231766:subnet/subnet-09b10ebd14209d020"  
dev-subnet-id  = "subnet-09b10ebd14209d020"  
dev-vpc-arn   = "arn:aws:ec2:me-central-1:535942231766:vpc/vpc-05764833e3ed15162"  
dev-vpc-id    = "vpc-05764833e3ed15162"  
@toobashafique065 [ /workspaces/Lab10 (main) ] $
```

Return ALL of the following output attributes from your VPC and Subnet resources:

- a) cidr\_block
- b) region
- c) tags.Name
- d) tags\_all

Expand output section in main.tf accordingly:

```
output "dev-vpc-cidr_block" { value =  
aws_vpc.development_vpc.cidr_block  
}
```

```
output "dev-vpc-region" { value =  
aws_vpc.development_vpc.region }
```

```
output "dev-vpc-tags_name" { value =  
aws_vpc.development_vpc.tags["Name"]  
}
```

```
output "dev-vpc-tags_all" { value =  
aws_vpc.development_vpc.tags_all  
}
```

```
output "dev-subnet-cidr_block" { value =  
aws_subnet.dev_subnet_1.cidr_block  
}
```

```
output "dev-subnet-region" { value =  
aws_subnet.dev_subnet_1.availability_zone  
}
```

```
output "dev-subnet-tags_name" { value =  
aws_subnet.dev_subnet_1.tags["Name"]  
}
```



```
output "dev-subnet-tags_all" { value =  
  
aws_subnet.dev_subnet_1.tags_all  
  
}
```

Run terraform apply and record outputs.

```
infrastructure.  
  
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.  
  
Outputs:  
dev-subnet-arn = "arn:aws:ec2:me-central-1:535942231766:subnet/subnet-09b10ebd14209d020"  
dev-subnet-cidr_block = "10.0.10.0/24"  
dev-subnet-id = "subnet-09b10ebd14209d020"  
dev-subnet-region = "me-central-1a"  
dev-subnet-tags_all = tomap({  
  "Name" = "subnet-1-dev"  
})  
dev-subnet-tags_name = "subnet-1-dev"  
dev-vpc-arn = "arn:aws:ec2:me-central-1:535942231766:vpc/vpc-05764833e3ed15162"  
dev-vpc-cidr_block = "10.0.0.0/16"  
dev-vpc-id = "vpc-05764833e3ed15162"  
dev-vpc-region = "me-central-1"  
dev-vpc-tags_all = tomap({  
  "Name" = "development"  
})  
dev-vpc-tags_name = "development"  
@toobashafique065 @ /workspaces/Lab10 (main) $ terraform output  
dev-subnet-arn = "arn:aws:ec2:me-central-1:535942231766:subnet/subnet-09b10ebd14209d020"  
dev-subnet-cidr_block = "10.0.10.0/24"  
dev-subnet-id = "subnet-09b10ebd14209d020"  
dev-subnet-region = "me-central-1a"  
dev-subnet-tags_all = tomap({  
  "Name" = "subnet-1-dev"  
})  
dev-subnet-tags_name = "subnet-1-dev"  
dev-vpc-arn = "arn:aws:ec2:me-central-1:535942231766:vpc/vpc-05764833e3ed15162"  
dev-vpc-cidr_block = "10.0.0.0/16"  
dev-vpc-id = "vpc-05764833e3ed15162"  
dev-vpc-region = "me-central-1"  
dev-vpc-tags_all = tomap({  
  "Name" = "development"  
})  
dev-vpc-tags_name = "development"
```

## Cleanup — Delete Resources & State Verification

Destroy all resources: `terraform destroy`

```

@toobashafique065 @ /workspaces/Lab10 (main) $ terraform destroy -auto-approve
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development_vpc: Refreshing state... [id=vpc-05764833e3ed15162]
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-0b8fa1602193cc7f7]
aws_subnet.dev_subnet_1_existing: Refreshing state... [id=subnet-088a3a5a00949bdba]
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-09b10ebd14209d020]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
- destroy

Terraform will perform the following actions:

# aws_subnet.dev_subnet_1 will be destroyed
- resource "aws_subnet" "dev_subnet_1" {
  - arn                                = "arn:aws:ec2:me-central-1:535942231766:subnet/subnet-09b10ebd14209d020" -> null
  - assign_ipv6_address_on_creation    = false -> null
  - availability_zone                  = "me-central-1a" -> null
  - availability_zone_id                = "mec1-az1" -> null
  - cidr_block                         = "10.0.10.0/24" -> null
  - enable_dns64                       = false -> null
  - enable_lni_at_device_index         = 0 -> null
  - enable_resource_name_dns_a_record_on_launch = false -> null
  - enable_resource_name_dns_aaaa_record_on_launch = false -> null
  - id                                 = "subnet-09b10ebd14209d020" -> null
  - ipv6_native                        = false -> null
  - map_customer_owned_ip_on_launch    = false -> null
  - map_public_ip_on_launch            = false -> null
}

```

Inspect state files:

cat terraform.tfstate

cat terraform.tfstate.backup

```

@toobashafique065 @ /workspaces/Lab10 (main) $ cat terraform.tfstate
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 41,
  "lineage": "085aa91c-daa5-b2aa-fd61-e66f10576ff1",
  "outputs": {},
  "resources": [],
  "check_results": null
}

@toobashafique065 @ /workspaces/Lab10 (main) $ cat terraform.tfstate.backup
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 36,
  "lineage": "085aa91c-daa5-b2aa-fd61-e66f10576ff1",
  "outputs": {
    "dev-subnet-arn": {
      "value": "arn:aws:ec2:me-central-1:535942231766:subnet/subnet-09b10ebd14209d020",
      "type": "string"
    },
    "dev-subnet-cidr_block": {
      "value": "10.0.10.0/24",
      "type": "string"
    },
    "dev-subnet-id": {
      "value": "subnet-09b10ebd14209d020",
      "type": "string"
    }
  },
}

```